

S-two recursive privacy blinding: disclosure summary

September 15, 2026

Status: Technical summary of the saved audit report, prepared for disclosure review. The original report remains in the research archive. This version summarizes evidence, impact, limitations, and remediation; it does not include the witness-recovery procedure or executable artifacts.

Audited source: starkware-libs/proving@cd7bc5f4697fb188a27e09f9242f1dd76df8afdc.

Affected area: The recursive privacy prover and its shared `add_zk_blinding` helper.

Finding

The saved audit reports a source-compatible valid toy circuit for which the disclosed blinding transcript determines a private witness on a specified class of public opening configurations. The circuit admits different private witnesses with the same public output. The reported algebraic result holds even when the masks are fresh and independent.

The issue concerns whether the helper can be used as a generic witness-hiding transformation. Adding random rows does not alone prove that the complete collection of disclosed evaluations hides the private part of a trace. The relevant condition concerns their combined information content and the way the masking randomness enters them.

Evidence and what it establishes

The original report records exact finite-field calculations showing that the public observation map distinguishes valid private witnesses in its stated toy circuit. It also records checks of finalization and padding compatibility. These are reported audit results; preparation of this summary did not rerun the recovery experiment or a native recursive verifier.

The audit separately analyzes opening configurations under an idealized challenge model. That calculation is not a theorem about the complete Fiat–Shamir transcript distribution of the deployed application. An exact conditional algebraic finding and a concrete end-to-end privacy claim require different evidence.

Impact and unresolved scope

The finding challenges an unrestricted claim that this helper hides witnesses for arbitrary valid circuits under the audited transcript interface. A privacy argument for the fixed recursive application

must therefore account for its actual witness relation, masking distribution, and complete transcript.

The saved report does not establish witness recovery from an accepted production privacy transaction. Two material questions remain: whether the demonstrated relation is admitted by the fixed recursive verifier, and whether the relevant opening event has a proved probability under the complete Fiat–Shamir process. No deployed secret or live service was accessed in the reported audit.

The report also identifies a separate randomness obligation: the audited path derives its masking seed from a proof commitment. Any privacy proof must justify the resulting distribution and its dependence on the witness; the mere presence of a seed is not such a justification.

Recommended remediation and validation

1. Specify the privacy property, supported witness relation, and complete transcript, including query and out-of-domain disclosures.
2. Establish a simulator argument or an equivalent information-hiding proof for that relation and transcript. Account for all disclosed projections together when choosing masking dimensions and placement.
3. Give the prover-randomness interface an explicit entropy and independence contract, and justify any deterministic seed derivation.
4. Add regression checks comparing valid witnesses with the same public statement, together with transcript-level leakage checks and proof/verifier round trips for supported component configurations.
5. Record affected versions, the implemented correction, and the security argument for the corrected configuration.

Relationship to the research paper

This privacy finding is separate from the repository’s list-decoding and proximity-gap results. Those results concern quantitative coding premises in soundness analyses. They do not establish the privacy finding, and this finding does not establish an end-to-end soundness failure or an improved coding-challenge submission.